When the Tool Layer Commoditizes: The Evolving Landscape of Software in Drug Discovery

Abstract

Commercial computational drug discovery software has traditionally bundled three sources of value: the user interface, the scientific models, and the software-specific user expertise. Large language models, coding agents, and tool-using scientific agents are beginning to unbundle this paradigm. Interfaces, dashboards, workflow orchestration, and substantial portions of scientific software can now be generated and adapted to the needs of a specific scientist, project, or organization, while increasingly sophisticated workflows in cheminformatics, quantum chemistry, molecular simulation, and molecular design can be automated and improved. We argue that this does not commoditize computational drug discovery itself; it shifts where differentiation resides. Easier and faster execution does little to increase the probability of success if the underlying predictive models are inaccurate. As the tool layer becomes cheaper and more accessible, advantage will increasingly accrue to better and more generalizable models, proprietary experimental data, and accumulated discovery intelligence encompassing expert judgment, design rationale, objectives, and organizational know-how. The transition is therefore from fixed software products toward adaptive discovery environments in which scientists interact directly with powerful models and agents. We use Molarium, an open-source browser-based molecular workbench developed over an intensive weekend by a drug-discovery scientist working with a coding agent, to illustrate how rapidly the software layer can now be reconstructed and customized.

Introduction

Successful computational drug discovery depends on applying the right method at the right time, with appropriate settings, assumptions, and scientific context. Historically, this has been a tedious, error-prone, and often poorly reproducible process confined to a relatively small number of computational specialists. Docking requires careful protein preparation, protocol selection, use of appropriate constraints, and critical evaluation of the resulting poses before they can inform design. Molecular dynamics adds requirements for system preparation, force-field assignment, simulation protocols, compute infrastructure, trajectory analysis, and visualization, while free-energy calculations introduce further complexity in perturbation design, sampling, convergence assessment, and alignment with experimental conditions. Similar layers of infrastructure surround cheminformatics, quantum chemistry, property prediction, molecular generation, and structural modeling. These workflows are often difficult to modify as projects evolve, and project-specific learnings can be challenging to incorporate quickly enough to influence ongoing decisions. Commercial scientific software has created substantial value by simplifying and

integrating many of these activities, but this convenience can come at the cost of flexibility, control, and rapid iteration. Graphical interfaces reduce technical barriers but constrain how problems are approached, while workflow systems improve consistency but can become rigid and difficult to adapt. The result has been an operating model in which computational expertise is often organized around mastery of specialized software rather than around the broader drug-discovery questions that the software is ultimately intended to address.

Large language models and coding agents are beginning to change this paradigm. A scientist with deep drug-discovery expertise but limited programming experience can increasingly describe an application, analysis, visualization, or workflow in natural language and have the software constructed around the scientific need. Tool-using agents extend this capability by operating existing methods, linking calculations, handling routine failures, and assembling results into forms that support decisions. The consequence is not simply faster software development, but a fundamental shift in how drug hunters interact with computational drug discovery tools: from learning and adapting to fixed software environments toward shaping computational workflows around the problem at hand.

We find it useful to divide the emerging computational environment into three layers. The tool layer includes interfaces, dashboards, visualization, data plumbing, software wrappers, job orchestration, and routine workflow logic. The model layer contains the scientific methods that determine predictive quality, including AI/ML models, force fields, quantum mechanics, free-energy methods, docking, protein structure prediction and co-folding, molecular representations, and property predictors. The discovery-intelligence layer comprises proprietary experimental data, project history, internal protocols, expert judgment, design rationale, organizational preferences, and business objectives. These layers remain tightly connected, but they are not commoditizing at the same rate.

Our central thesis is that the cost of building interfaces and operating computational tools is falling rapidly, while the scientific challenges in the model layer remain substantial. At the same time, agents create new opportunities to capture and operationalize discovery intelligence that has historically been difficult to formalize or scale. As a result, competitive advantage in computational drug discovery is likely to shift away from mastery of fixed software environments and toward better models, richer proprietary data, and more effective scientific decision-making.

From fixed applications to adaptive scientific environments

Traditional graphical interfaces solved an important problem. They translated complex scientific software into a finite set of operations that researchers could learn and apply reproducibly. However, a fixed interface necessarily reflects decisions made in advance about what information matters, which functions should be exposed, and how users are expected to move through a workflow. Those decisions must accommodate many targets, modalities, organizations, and stages of discovery, and therefore inevitably involve compromise.

Drug discovery itself is much less standardized. A fragment-screening program may organize decisions around binding-site interactions, ligand efficiency, crystallographic waters, and vectors

for growth. A macrocycle program may care more about conformational ensembles, intramolecular hydrogen bonds, permeability, and the relationship between solution and bound states. A mature lead-optimization program may need to integrate potency, selectivity, clearance, solubility, safety liabilities, structural hypotheses, free-energy predictions, and synthetic timelines. Even within the same program, the most useful interface can change substantially as the project evolves.

Coding agents make it increasingly possible to invert the conventional relationship between the scientist and the software. Rather than adapting the scientific question to the structure of an existing application, the scientist can describe the decision that needs to be made and construct the interface around that purpose. A medicinal chemist could request a view of the current series organized by permeability and clearance, overlaid with binding free-energy predictions and structural hypotheses. If a new experimental result changes the relevant design question, the interface can be modified immediately rather than waiting for a future product release.

This does not imply that graphical interfaces become unimportant. Molecular design remains highly visual, and good interfaces are essential for reasoning across chemical structures, three-dimensional interactions, experimental data, and competing hypotheses. What changes is the scarcity of the interface itself. A graphical environment can increasingly become an adaptive representation of the current scientific problem rather than a fixed product that defines how the problem must be approached.

The commercial implications are significant. Historically, substantial value could be created by combining multiple scientific capabilities behind a polished interface and providing the workflow infrastructure required to connect them. That integration remains useful, particularly in large organizations where security, validation, deployment, and support matter. However, generic dashboards, workflow plumbing, and simple wrappers around established methods are becoming considerably easier to reproduce. The more rapidly this occurs, the more commercial differentiation will depend on scientific capabilities that are difficult to reconstruct.

Agents are changing who can operate computational methods

The same disruption is beginning to affect the operation of scientific workflows. Computational chemistry has traditionally required specialists partly because the procedures surrounding the calculations are complicated. A molecular dynamics simulation requires more than selecting a trajectory length. Protein preparation, protonation states, cofactors, parameterization, solvation, equilibration, restraints, quality control, and analysis all require decisions. Quantum chemistry has analogous choices around methods, basis sets, electronic states, convergence, and interpretation. Free-energy calculations require decisions about perturbation networks, sampling, diagnostics, and whether a particular transformation is scientifically reasonable.

Recent chemistry agents provide early evidence that a growing fraction of this operational knowledge can be encoded. ChemCrow demonstrated that a language model could coordinate chemistry-specific tools and external information sources to solve multistep tasks. CACTUS

similarly connects language-model reasoning to established cheminformatics capabilities rather than asking the language model to reproduce those calculations internally. More specialized systems such as MDCrow, ChemGraph, and El Agente have extended this concept to molecular dynamics and quantum chemistry workflows, including task decomposition, execution, analysis, and error recovery.

These demonstrations should not be interpreted as evidence that current agents generally outperform experienced computational scientists. Published benchmarks are necessarily bounded and typically cleaner than difficult prospective drug-discovery problems. Real projects contain uncertain structures, unusual chemistries, contradictory experimental observations, poorly understood sampling problems, and circumstances in which the correct computational strategy is itself unclear. Nevertheless, the important trend is already visible: much of the operational expertise required to execute well-defined workflows can be encoded as reusable agent skills.

This has practical advantages. Agents can apply protocols systematically, examine more alternatives than a busy scientist may consider, maintain complete execution records, monitor calculations asynchronously, and repeat workflows without fatigue. They can work overnight and over weekends, and they can perform large numbers of routine operations at marginal costs that are very different from specialist labor or conventional per-user software licensing. For sufficiently well-defined tasks, the question increasingly becomes not whether an agent can operate the method, but whether the method itself is good enough to support the decision.

Molarium as a case study of the tool layer

Molarium provides a useful illustration of this changing boundary. It is a local-first molecular workbench that runs primarily in a web browser and integrates molecular visualization and editing with chemical perception, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and protein prediction. The current implementation draws on established open-source and scientific components including RDKit, OpenFF, ANI-2x, and OpenFold, while also incorporating browser-native numerical functionality.

The particular set of capabilities is less important than the development process. Molarium began as a request for a more useful way to inspect and manipulate molecules. Over an intensive weekend, interaction between an experienced drug-discovery scientist and a coding agent produced a substantial scientific application. When a molecular edit produced an implausible geometry, the problem was challenged and the implementation changed. When an energy appeared questionable, a reference calculation was introduced. When a workflow needed to preserve chemical and structural context, the interface and execution path evolved accordingly.

This development model is qualitatively different from the conventional software cycle in which scientific requirements are translated into specifications, prioritized against other features, implemented by a software team, tested, released, and eventually returned to the scientist. The distance between identifying a scientific need and implementing the corresponding software can now become extremely short.

Equally important is what Molarium did not recreate. The coding agent did not regenerate the accumulated scientific knowledge contained in force-field parameter sets, trained model weights, or experimental validation. Numerical agreement with a reference implementation can establish that a particular piece of software is behaving as intended; it does not establish that the underlying scientific model accurately describes an arbitrary molecular system. A model can be implemented perfectly and still make the wrong prediction.

Molarium therefore illustrates a broader distinction. Software implementation is becoming easier to reproduce. Scientific validity is not.

Automation alone does not solve the prediction problem

The ability to automate computational workflows creates an understandable temptation to equate greater throughput with better drug discovery. In practice, the two are only loosely coupled. An agent may prepare a protein, dock thousands of molecules, test alternative protonation states, run molecular dynamics, initiate free-energy calculations, retrieve property predictions, and organize the results in an elegant dashboard. The workflow can be technically impressive while the resulting prediction remains wrong.

The limitations arise from the scientific models themselves. A docking workflow cannot reliably identify the correct pose if the scoring function does not distinguish it from plausible alternatives. A molecular dynamics simulation cannot recover the correct thermodynamics if the underlying energy function is inappropriate for the system. A free-energy calculation remains dependent on force-field quality, sampling, protonation states, structural hypotheses, and the relationship between the simulated and experimental conditions. An ADME model may produce precise numerical predictions while operating well outside the chemistry represented in its training data.

This problem is particularly acute in drug discovery because novelty is usually the objective. Medicinal chemists intentionally modify structures to move into chemical space that has not been measured before. Targets with the greatest therapeutic interest often have the least historical chemical data. Biological measurements are sparse, heterogeneous, and dependent on experimental context. Generalization therefore matters at least as much as retrospective fit.

Agents can make weak models easier to use, but they cannot make them more predictive simply by invoking them more systematically. Indeed, automation can magnify model failure. A human may run a small number of carefully selected calculations, whereas an agent may generate thousands of internally consistent predictions. If the model has a systematic error, scale can create a larger volume of confidently incorrect output.

The appropriate benchmark for an agentic computational system is therefore not the number of calculations executed, the number of molecules scored, or the amount of human labor removed. The relevant question is whether the system improves the decisions that determine which molecules are synthesized, which experiments are performed, and which programs continue.

The model layer becomes more important as the tool layer becomes easier

As access to computation becomes easier, differences in model quality become more consequential. When running a sophisticated calculation requires substantial specialist effort, simply having the capability can create competitive advantage. When the same calculation can be initiated through an agent by any appropriately trained drug designer, the important distinction becomes whether one model predicts the relevant molecular behavior better than another.

This principle applies across the computational stack. For docking, the important issue is pose and ranking accuracy. For molecular dynamics, it is whether the physical model and sampling capture the relevant states and transitions. For free-energy calculations, structural assumptions, sampling, force fields, and uncertainty remain central. For protein structure prediction and co-folding, average benchmark performance is less important than whether the model generalizes to the particular interaction or conformational state relevant to the program. For ADME models, the central question is whether predictions remain reliable as medicinal chemistry moves into new chemical space.

The implications for scientific software are straightforward. A commercial product that contains a genuinely differentiated predictive model can remain highly valuable even if the interface surrounding that model becomes easy to reproduce. In fact, easier interfaces may increase the value of the model by making it accessible to more users and allowing it to be incorporated into more workflows. Conversely, the moat around an application whose primary differentiation lies in a convenient interface around otherwise reproducible methods is likely to shrink.

We therefore expect increasing separation between the scientific capability and the environment through which it is accessed. A project-specific internal interface may call a mixture of commercial models, open-source methods, and proprietary calculations. The user may not need to know which software package historically owned each capability. What matters is whether the model is appropriate, validated, and useful for the decision being made.

Better molecular representations may matter more than better orchestration

Most current scientific agents operate in what can be considered tool space. They choose among discrete operations such as docking, molecular dynamics, free-energy calculations, similarity searching, quantum chemistry, or property prediction. This orchestration can create substantial efficiency, but it does not necessarily improve the representation of the molecular system on which the underlying models depend.

Representation places a fundamental limit on what a model can learn. Molecules can be encoded as strings, fingerprints, two-dimensional graphs, three-dimensional structures, electronic descriptors, or learned embeddings. Each representation captures some aspects of molecular

reality while discarding others. Consequently, increasingly capable agents cannot fully overcome limitations imposed by an inadequate representation.

Learned latent representations provide a complementary direction. The seminal work of Gómez-Bombarelli and colleagues demonstrated that discrete molecules could be mapped into continuous latent spaces in which property prediction, interpolation, and molecular optimization become possible. The broader significance of this concept is that molecular design need not be restricted to enumerating structures and scoring them independently. If the geometry of a learned molecular space reflects meaningful chemistry, one can begin to navigate that space toward regions associated with desired properties.

This distinction separates two related but different opportunities. In tool space, an agent asks which calculation should be run next. In a sufficiently accurate latent representation, a model can begin to ask in which direction a molecule should move. The former makes existing computational workflows easier to execute. The latter has the potential to make molecular design itself more efficient by providing a continuous representation in which multiple properties and interventions can be optimized.

The most powerful discovery systems will likely combine these capabilities. Agents can orchestrate physical simulations, structural models, property predictions, experimental databases, and generative methods, while improved molecular representations provide a common space for prediction and optimization. Ultimately, such systems may support inverse molecular design, where the starting point is a desired molecular profile and the computational system searches for molecules predicted to satisfy it.

The scientific requirements are substantial. A useful representation must generalize to new chemistry, capture the molecular features relevant to the properties being optimized, reflect the environmental dependence of molecular behavior, and provide meaningful uncertainty as the system moves away from familiar regions. These are model-layer challenges and will not disappear as software becomes easier to build.

Proprietary data become more important when their context is preserved

As generic computational tools become easier to access, proprietary experimental data provide an increasingly obvious source of differentiation. Drug-discovery organizations generate enormous amounts of information that never appears in public datasets, including unsuccessful compounds, weak or contradictory assay results, structural and biophysical measurements, ADME data, pharmacokinetics, target-specific observations, and chemical series that were abandoned for reasons not apparent from their final numerical values.

The value of these data depends strongly on context. A potency measurement depends on assay format, protein construct, substrate concentration, cell background, incubation time, and other experimental variables. A permeability measurement depends on the assay system and protocol. A structural observation may represent one state within a broader conformational ensemble. A

compound labeled inactive may have been insoluble, unstable, poorly exposed, or tested against an inappropriate biological hypothesis.

For this reason, proprietary data should not be treated simply as a larger private version of a public training set. The most informative datasets preserve experimental provenance, uncertainty, molecular state, biological context, and the scientific reason the experiment was performed. This context becomes particularly important when data are used by agents or learning systems that otherwise have no way to distinguish a failed compound from a failed hypothesis.

Negative data may be especially valuable. Public information strongly favors successful experiments and active compounds, whereas internal discovery programs accumulate large numbers of unsuccessful designs and abandoned ideas. These observations help define the boundaries of chemical and biological space and can provide information that is difficult for external models to recover.

Discovery intelligence may become the deepest moat

Even proprietary data do not capture the full knowledge accumulated during a drug-discovery program. Many of the most consequential insights are expressed as judgments rather than measurements. A medicinal chemist may decide not to synthesize a molecule because a related motif consistently creates a metabolic liability. A computational scientist may distrust an otherwise favorable prediction because the proposed modification changes the protonation equilibrium or forces a poorly sampled conformational transition. A project team may intentionally select a compound with less favorable predicted properties because it provides the most informative experiment for distinguishing between competing hypotheses.

These decisions encode causal reasoning, project objectives, risk tolerance, and scientific taste. Drug design is intrinsically multiobjective, and the relative importance of potency, permeability, clearance, selectivity, synthetic accessibility, novelty, uncertainty, and speed changes across programs and stages of discovery. A generic model cannot infer all of these priorities from molecular structures and assay values alone.

Agents provide a mechanism for turning some of this accumulated knowledge into reusable infrastructure. A protein-preparation skill can encode an organization's preferred treatment of protonation states, crystallographic waters, alternate conformations, and ambiguous residues. A free-energy skill can specify accepted transformations, convergence criteria, required diagnostics, and conditions that trigger additional sampling. A compound-selection skill can combine calculated and measured properties with the current target product profile and record why particular molecules were selected or rejected.

These skills represent more than automation. Their execution may be generic, but their content can be highly proprietary. Two organizations using the same underlying foundation model, molecular dynamics engine, and public structural data could make very different decisions because their agents have access to different experimental histories, encode different scientific practices, and optimize against different objectives.

We refer to this broader asset as discovery intelligence: the combination of proprietary data, scientific context, validated models, project history, design rationale, organizational preferences, and accumulated expert judgment. Historically, much of this information remained fragmented across databases, presentations, electronic laboratory notebooks, emails, meetings, and scientists' memories. Agentic systems provide an opportunity to capture it and allow it to compound over successive discovery cycles.

The role of the scientist becomes more important

There is an apparent paradox in increasingly capable scientific software. As the mechanics of computation become easier, the scientific judgment of the user becomes more important rather than less. When software is difficult to operate, a meaningful fraction of expertise is spent learning interfaces, preparing files, configuring jobs, and troubleshooting workflows. As these tasks become automated, expertise can shift toward choosing the right question, selecting the appropriate model, identifying implausible results, and deciding what should be done next.

Molarium provided a simple example. The coding agent could implement requested functionality quickly, but a scientist still had to recognize that a molecular geometry was chemically implausible, question an energy scale, identify the appropriate reference calculation, and decide whether the discrepancy mattered. The agent reduced the distance between scientific judgment and software implementation. It did not replace the judgment.

The same principle should apply to computational drug-discovery organizations. Medicinal chemists and other drug designers can increasingly interact directly with computational capabilities without becoming specialists in every underlying implementation. A scientist should be able to interrogate a molecular dynamics trajectory, compare free-energy hypotheses, or build a project-specific analysis without learning a collection of separate software environments.

This does not diminish the importance of computational scientists. It changes where their expertise creates the most leverage. Rather than spending substantial time operating routine workflows, building repetitive dashboards, or serving as intermediaries between software and drug designers, specialists can focus more heavily on model development, molecular physics, validation, uncertainty, difficult sampling problems, novel representations, and the scientific situations in which established methods fail. Their expertise can then be encoded into workflows that scale across the organization.

Implications for commercial drug discovery software

Commercial scientific software will remain important. Reliable implementations, enterprise security, high-performance computing, provenance, validation, regulatory requirements, maintenance, technical support, and integration with organizational data systems remain difficult problems. The availability of coding agents does not remove these requirements.

However, the composition of value is likely to change. Fixed interfaces, generic workflow orchestration, simple integrations between established methods, and routine analysis scripts are

becoming easier to reproduce. In contrast, scientifically superior models, proprietary training data, validated parameterization, prospective performance, uncertainty quantification, specialized algorithms, scalable infrastructure, and trusted scientific support remain difficult to replicate.

This may also change how software is consumed. Per-user licensing is natural when the application and its interface define the product. It becomes less obvious when scientists and agents interact with scientific capabilities through dynamically generated internal environments. Commercial methods may increasingly be exposed as composable models or services, with organizations constructing their own interfaces and workflows around them.

Open-source software accelerates this transition by providing high-quality scientific primitives that can be inspected, combined, modified, and invoked by agents. The consequence is not that commercial software loses value, but that commercial differentiation must increasingly reside in something deeper than having assembled a convenient collection of functions.

Validation becomes more important as generation becomes easier

Rapid software generation also increases the importance of distinguishing implementation validation from scientific validation. An agent may correctly reproduce an algorithm and pass a suite of numerical tests. That establishes that the implementation behaves as intended under the tested conditions. It does not establish that the underlying model accurately predicts the molecular system of interest.

A third level is even more important in drug discovery: decision validation. A computational approach ultimately creates value when its use improves the choice of compounds, experiments, or programs. An accurate calculation that does not affect a decision may have little practical impact, whereas a moderately accurate model that consistently eliminates poor compounds or identifies unexpected opportunities can create substantial value.

Agentic systems should therefore be evaluated against the same standard. The number of autonomous workflows executed is not itself evidence of scientific progress. Neither is the number of compounds designed or the amount of human effort removed. The relevant questions are whether the predictions generalize prospectively, whether uncertainty is informative, whether the system changes experimental choices, and whether those choices improve the probability of producing better molecules.

This is particularly important because agents can generate large volumes of coherent scientific output. Fluency and systematic execution can create an appearance of confidence that is not necessarily supported by the underlying model. As the cost of generating analyses approaches zero, rigorous validation and scientific skepticism become increasingly valuable.

Conclusions

The computational drug discovery software landscape is beginning to unbundle. Scientific models, graphical interfaces, workflow implementation, and specialist operating expertise have historically been tightly coupled because each was expensive to develop and maintain. Large language models, coding agents, open-source scientific software, and tool-using agents are weakening that coupling. Interfaces can increasingly adapt to individual programs, substantial portions of the tool layer can be generated when needed, and routine computational procedures can be encoded as reusable skills.

This development does not make computational drug discovery itself a commodity. The difficult problems remain scientific. Molecular behavior is still governed by physics and chemistry. Force fields contain approximations, simulations have sampling limitations, learned models have applicability domains, biological measurements remain sparse and contextual, and molecular representations determine what relationships can be learned. Making a calculation easier to run is valuable, but it is fundamentally different from making the calculation more accurate.

We therefore expect the locus of differentiation to move toward the model and discovery-intelligence layers. Better representations of molecular systems, more accurate and generalizable models, well-calibrated uncertainty, proprietary experimental evidence, and prospective validation should become increasingly important as access to computation broadens. Organizations that capture their scientific history, internal practices, design rationale, and objectives can create an additional layer of proprietary discovery intelligence that compounds over time.

The organizational implications are equally important. Computational capability can increasingly move directly into the hands of the drug hunters making molecular-design decisions, while computational specialists focus on the hardest scientific problems and encode their expertise into systems that the broader organization can use. Commercial software vendors can continue to create substantial value, but durable differentiation will increasingly depend on scientific assets that are difficult to reproduce rather than on the complexity of the interface surrounding them.

The important transition is therefore not from scientists to agents, nor from commercial software to internally generated applications. It is from a world in which scientific capability is constrained by access to specialized software toward one in which the software increasingly adapts to the scientific problem.

When the tool layer commoditizes, the remaining sources of advantage become clearer: better models of molecular reality, better proprietary learning loops, and better drug hunters empowered to use them.